Compêndio de Questões de Provas

Agrupamento por Métodos de Resolução e Soluções Detalhadas

Disciplina: Projeto e Análise de Algoritmos (PAA) — UFAM — Compilação de 8 Avaliações

Abstract

Este documento compila, organiza e resolve todas as questões extraídas do conjunto de 8 provas da disciplina de **Projeto e Análise de Algoritmos (PAA)**. As questões foram agrupadas em 5 categorias metodológicas fundamentais. No início de cada grupo, é apresentado um **passo a passo geral de resolução**, seguido das questões pertencentes a esse grupo com suas respectivas resoluções detalhadas passo a passo.

Contents

1 Grupo 1: Notação Assintótica, Limites e Análise Verdadeiro/Falso 2

2 Grupo 2: Análise de Laços Iterativos e Contagem de Operações 5

3 Grupo 3: Algoritmos Recursivos e Equações de Recorrência 8

4 Grupo 4: Programação Dinâmica 11

5 Grupo 5: Teoria de Algoritmos, Estruturas de Dados e Estimativa Empírica 13

1 Grupo 1: Notação Assintótica, Limites e Análise Verdadeiro/Falso

Guia Passo a Passo Geral — Notação Assintótica e V/F

Para responder a questões que envolvem classificação de funções, notações O, Ω, Θ, o e afirmações de Verdadeiro ou Falso:

1. **Identificar a Notação Pedida**:
   - **Cota Superior Assintótica (O)**: $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$ ($c > 0, n_0 > 0$).
   - **Cota Inferior Assintótica (Ω)**: $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$.
   - **Cota Justa Assintótica (Θ)**: $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.
   - **Cota Estrita (o)**: $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.
2. **Aplicar Prova por Definição ou Limites**:
   - Escolher constantes c e n_0 explícitas para provar limites superiores/inferiores.
   - Usar a Regra de L'Hôpital se necessário para calcular $\lim_{n\to\infty} \frac{f(n)}{g(n)}$.
3. **Lembras das Hierarquias de Crescimento**:

$$\Theta(1) \ll \Theta(\log n) \ll \Theta(n) \ll \Theta(n \log n) \ll \Theta(n^k) \ll \Theta(c^n) \ll \Theta(n!)$$

4. **Atenção a Armadilhas Conceituais**:
   - $O(g(n))$ aceita cotas frouxas (ex: $4n = O(n^3)$ é **Verdadeiro**).
   - Subtração de ordens assintóticas como $\Theta(n) - \Theta(n/2)$ não resulta em números arbitrários como $\Omega(1.5n)$.

Questão 1.1 *Prova 2009-A — Q1-A*

Se $f(n) = 2$, então $f(n)$ é $O(2^n)$? Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Definição de O**: $f(n) \in O(2^n)$ se existirem constantes $c > 0$ e $n_0 > 0$ tais que $2 \leq c \cdot 2^n$ para todo $n \geq n_0$.
2. **Escolha das Constantes**: Escolhendo $c = 1$ e $n_0 = 1$, a desigualdade torna-se $2 \leq 1 \cdot 2^n$.
3. **Verificação**: Para $n = 1$, $2 \leq 2$ (satisfeito). Para todo $n \geq 1$, $2^n \geq 2$, logo a condição se mantém.
4. **Conclusão**: Afirmação **VERDADEIRA**. Embora $O(2^n)$ seja uma cota superior frouxa (já que a cota justa é $\Theta(1)$), a definição matemática de Big-O é plenamente satisfeita.

Questão 1.2 *Prova 2009-A — Q1-B*

Todo programa que tem custo $o(n)$ também tem custo $O(n)$? Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Definição de** $o(n)$: Significa que a função cresce estritamente mais devagar que n, ou seja, $\lim_{n\to\infty} \frac{f(n)}{n} = 0$.
2. **Implicação para** $O(n)$: Pela definição de limite, para qualquer $\epsilon > 0$, existe n_0 tal que $f(n) \leq \epsilon \cdot n$ para todo $n \geq n_0$. Escolhendo $c = \epsilon$, atende-se exatamente à definição de $f(n) \in O(n)$.
3. **Relação de Conjuntos**: O conjunto $o(n)$ é um subconjunto próprio de $O(n)$ ($o(n) \subset O(n)$).
4. **Conclusão**: Afirmação **VERDADEIRA**.

Questão 1.3 *Prova 2013-A — Q1-A*

Se $f(n) = n + n + n + n$, então $f(n)$ é $O(n^3)$? Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Simplificação**: $f(n) = 4n$.
2. **Aplicação da Definição**: Desejamos verificar se $4n \leq c \cdot n^3$ para $n \geq n_0$.
3. **Escolha de Constantes**: Para $c = 4$ e $n_0 = 1$, temos $4n \leq 4n^3 \iff 1 \leq n^2$, o que é verdadeiro para todo $n \geq 1$.
4. **Conclusão**: Afirmação **VERDADEIRA**.

Questão 1.4 *Prova 2013-A — Q1-B*

Se $g(n) = n^{31.2}$, então $g(n)$ é $\Omega(n^{31.1})$? Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Definição de** Ω: $g(n) = \Omega(n^{31.1})$ se $n^{31.2} \geq c \cdot n^{31.1}$ para $n \geq n_0$.
2. **Manipulação Álgebrica**: Dividindo ambos os lados por $n^{31.1}$, obtemos $n^{0.1} \geq c$.
3. **Análise do Limite**: Como $n^{0.1}$ cresce ilimitadamente com n, para $c = 1$ e $n_0 = 1$, $n^{0.1} \geq 1$ vale para todo $n \geq 1$.
4. **Conclusão**: Afirmação **VERDADEIRA**.

Questão 1.5 *Prova 2018 — Q1-A*

Um algoritmo de hash por encadeamento tem pior caso assintótico pior que o de uma busca em um vetor ordenado, tendo custo linear no pior caso. Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Análise do Hash por Encadeamento**: No pior caso, todas as n chaves sofrem colisão e são inseridas na mesma lista encadeada. Buscar um elemento exige percorrer a lista inteira, resultando em custo $\Theta(n)$ (linear).

2. **Análise do Vetor Ordenado**: A busca em vetor ordenado utiliza busca binária, com custo no pior caso de $\Theta(\log n)$.
3. **Comparação Assintótica**: Como $\Theta(n)$ cresce estritamente mais rápido que $\Theta(\log n)$, o pior caso do hash é assintoticamente pior.
4. **Conclusão**: Afirmação **VERDADEIRA**.

Questão 1.6 *Prova 2018 — Q1-B*

Se $f(n) = \Theta(n) - \Theta(n/2)$, então $f(n)$ é $\Omega(1.5n)$? Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Análise da Subtração de Conjuntos Assintóticos**: A notação $\Theta(n)$ representa uma classe de funções $c_1 n \leq f_1(n) \leq c_2 n$.
2. **Contra-exemplo**: Seja $f_1(n) = n \in \Theta(n)$ e $f_2(n) = n/2 \in \Theta(n/2)$. A diferença é $f(n) = n - n/2 = 0.5n$.
3. **Verificação da Cota Inferior**: $0.5n$ não é $\Omega(1.5n)$, pois $0.5n < 1.5n$ para todo $n > 0$.
4. **Conclusão**: Afirmação **FALSA**.

Questão 1.7 *Prova 2018 — Q1-C*

Uma função de custo $n^2 + n$ não é $\Theta(n)$? Indique se é Verdadeira ou Falsa e justifique.

Resolução Passo a Passo:

1. **Cálculo do Termo Dominante**: Para n^2+n, o termo dominante é n^2, portanto $n^2+n \in \Theta(n^2)$.
2. **Verificação do Limite**: $\lim_{n\to\infty} \frac{n^2+n}{n} = \lim_{n\to\infty}(n+1) = \infty$.
3. **Implicação**: A função cresce estritamente mais rápido do que qualquer função linear $\Theta(n)$.
4. **Conclusão**: Afirmação **VERDADEIRA**.

Questão 1.8 *prova1-2024-ES-Substitutiva — Q1*

Indique a classe Θ e classifique as equações em Polinomiais (P) ou Exponenciais (E):

$$\begin{aligned} &\text{I)} \quad T(n) = 2T(n-2) + 1 \quad (n > 0), \quad T(0) = 1 \\ &\text{II)} \quad T(n) = 9T(n/3) + n \quad (n > 1), \quad T(1) = 1 \\ &\text{III)} \quad T(n) = 3T(n/3) + n \quad (n > 1), \quad T(1) = 1 \end{aligned}$$

Resolução Passo a Passo:

1. **Item I**: Recorrência com subtração. A árvore de recursão possui profundidade $n/2$ e fator de ramificação 2. O número total de nós é $\sum_{k=0}^{n/2} 2^k = 2^{n/2+1} - 1 = \Theta(2^{n/2}) = \Theta((\sqrt{2})^n)$. Classe: $\mathbf{\Theta(2^{n/2})}$, Tipo: **Exponencial (E)**.
2. **Item II**: Teorema Mestre com $a = 9, b = 3, f(n) = n$. $n^{\log_3 9} = n^2$. Como $f(n) = n = O(n^{2-\epsilon})$ ($\epsilon = 1$), aplica-se o Caso 1. Classe: $\mathbf{\Theta(n^2)}$, Tipo: **Polinomial (P)**.
3. **Item III**: Teorema Mestre com $a = 3, b = 3, f(n) = n$. $n^{\log_3 3} = n^1$. Como $f(n) = n = \Theta(n^1)$, aplica-se o Caso 2. Classe: $\mathbf{\Theta(n \log n)}$, Tipo: **Polinomial (P)**.

2 Grupo 2: Análise de Laços Iterativos e Contagem de Operações

Guia Passo a Passo Geral — Análise de Laços Iterativos

Para calcular a função de custo exata e a classe Θ de laços de repetição (FOR / WHILE):

1. **Analisar a Variável de Controle do Laço Externo**:
   - Passo incremental ($x = x + 1$): O laço executa n iterações.
   - Passo multiplicativo ($x = x * 2$): O laço executa $\lceil \log_2 n \rceil$ iterações.
2. **Analisar o Laço Interno em Função do Externo**:
   - Se o limite superior do laço interno for independente (ex: $y < n$), a contagem total é o produto simples.
   - Se o limite depender de x (ex: $y < x$), expressar o custo total como um somatório:

$$C(n) = \sum_{x} (\text{número de iterações do laço interno para aquele } x)$$

3. **Resolver o Somatório**:
   - Soma de Progressão Geométrica: $\sum_{k=0}^{m} 2^k = 2^{m+1} - 1$.
   - Soma de Progressão Aritmética: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.
4. **Extrair a Classe de Complexidade** Θ: Identificar o termo de maior ordem.

Questão 2.1 *Prova 2009-A — Q3-B*

Apresente a função de custo e a classe de complexidade para determinar o número de vezes que a operação `faz_algo` é executada:

```
VERIFICA_ALGO(int n) {
    FOR(x=1; x < n; x=x*2) {
        FOR(y=0; y < x; y++) { faz_algo; }
    }
}
```

Resolução Passo a Passo:

1. **Comportamento do Laço Externo**: x assume os valores $1, 2, 4, 8, \ldots, 2^k$, onde $2^k < n \implies k \leq \lfloor \log_2(n-1) \rfloor$.
2. **Comportamento do Laço Interno**: Para cada valor de x, o laço de y executa exatamente x vezes.
3. **Montagem do Somatório**:

$$C(n) = \sum_{k=0}^{\lfloor \log_2(n-1) \rfloor} 2^k$$

4. **Cálculo da Soma PG**: Usando a fórmula da soma da PG com razão 2:

$$C(n) = 2^{\lfloor \log_2(n-1) \rfloor + 1} - 1$$

5. **Análise Assintótica**: Para n potência de 2, $C(n) = 2 \cdot n - 1$.
6. **Resultado**: Função de Custo: $C(n) = 2^{\lfloor \log_2(n-1) \rfloor + 1} - 1$, Classe: $\mathbf{\Theta(n)}$.

Questão 2.2 — *Prova 2013-A — Q4*

Apresente a função de custo e a classe de complexidade para determinar o número de vezes que `faz_algo` é executada:

```
VERIFICA_ALGO(int n) {
    FOR(x = 0; x < n; x++) { faz_algo; }
    FOR(x = 0; x < n; x++) {
        FOR(y = 1; y < n; y = y*2) { faz_algo; }
    }
}
```

Resolução Passo a Passo:

1. **Primeiro Bloco**: Um único laço de 0 a $n-1$. Custo $= n$.
2. **Segundo Bloco**: Laço externo roda n vezes. Laço interno varia y dobrando $(1, 2, 4, \cdots < n)$, executando $\lceil \log_2 n \rceil$ vezes. Custo do segundo bloco $= n \cdot \lceil \log_2 n \rceil$.
3. **Custo Total**:
$$C(n) = n + n\lceil \log_2 n \rceil = n(1 + \lceil \log_2 n \rceil)$$
4. **Resultado**: Função de Custo: $C(n) = n(1 + \lceil \log_2 n \rceil)$, Classe: $\mathbf{\Theta(n \log n)}$.

Questão 2.3 — *Prova 2018 — Q5-A*

Apresente a função de custo do algoritmo abaixo:

```
VERIFICA_ALGO(int n) {
    FOR(x=0; x < log(n); x++) {
        FOR(y=0; y < n; y++) { faz_algo; }
    }
}
```

Resolução Passo a Passo:

1. **Laço Externo**: Roda $\lfloor \log_2 n \rfloor$ vezes (incremento unitário).
2. **Laço Interno**: Roda n vezes para cada iteração do laço externo.
3. **Custo Exato**: $C(n) = \lfloor \log_2 n \rfloor \cdot n$.
4. **Resultado**: Classe de Complexidade: $\mathbf{\Theta(n \log n)}$.

Questão 2.4 — *prova1-2024-ES-Substitutiva — Q2*

Apresente a classe de complexidade Θ do algoritmo detalhando o cálculo feito:

```
int f(int v[], int n) {
    for(int x=0; x < n; x++) {
        for(int y = 0; y <= n; y=y+2) { printf("%d", x*y); }
```

```
    }
}
```

Resolução Passo a Passo:

1. **Laço Externo**: Roda n vezes ($x = 0, 1, \ldots, n-1$).
2. **Laço Interno**: Varia y de 0 até n de 2 em 2 ($0, 2, 4, \ldots$). O número de iterações é $\lfloor \frac{n}{2} \rfloor + 1$.
3. **Somatório do Custo**:

$$C(n) = \sum_{x=0}^{n-1} \left(\lfloor \frac{n}{2} \rfloor + 1 \right) = n \left(\lfloor \frac{n}{2} \rfloor + 1 \right) = \frac{n^2}{2} + O(n)$$

4. **Resultado**: Classe de Complexidade: $\mathbf{\Theta(n^2)}$ (Quadrático).

Questão 2.5 *prova1-2026-Férias — Q2*

Apresente a classe de complexidade Θ do algoritmo detalhando o cálculo feito:

```
int f(int v[], int n) {
    for(int x=1; x < n; x=x*2) {
        for(int y = 0; y < 10*n; y++) { printf("%d", x*y); }
    }
}
```

Resolução Passo a Passo:

1. **Laço Externo**: x duplica a cada iteração ($1, 2, 4, \cdots < n$), executando $\lceil \log_2 n \rceil$ vezes.
2. **Laço Interno**: Roda $10n$ vezes independentemente do valor de x.
3. **Custo Total**: $C(n) = 10n \cdot \lceil \log_2 n \rceil$.
4. **Resultado**: Classe de Complexidade: $\mathbf{\Theta(n \log n)}$.

3 Grupo 3: Algoritmos Recursivos e Equações de Recorrência

Guia Passo a Passo Geral — Equações de Recorrência

Para determinar a função de custo exata e/ou a classe Θ de funções recursivas:

1. **Formular a Recorrência** $T(n)$:
   - Contar o trabalho interno realizado na chamada atual $\rightarrow f(n)$.
   - Somar o custo das chamadas recursivas $aT(n/b)$ ou $aT(n-c)$.
   - Definir os casos base $T(0)$ ou $T(1)$.
2. **Escolher a Técnica de Resolução**:
   - **Teorema Mestre** (para $T(n) = aT(n/b) + f(n)$): Calcular $n^{\log_b a}$ e comparar com $f(n)$:
     (a) **Caso 1**: $f(n) = O(n^{\log_b a - \epsilon}) \implies T(n) = \Theta(n^{\log_b a})$.
     (b) **Caso 2**: $f(n) = \Theta(n^{\log_b a}) \implies T(n) = \Theta(n^{\log_b a} \log n)$.
     (c) **Caso 3**: $f(n) = \Omega(n^{\log_b a + \epsilon})$ e $af(n/b) \leq cf(n) \implies T(n) = \Theta(f(n))$.
   - **Método da Iteração / Substituição** (para recorrências por subtração $T(n) = T(n-c)+g(n)$): Expandir a equação para k passos, identificar o padrão geral, igualar $n-kc = 0$ e resolver o somatório resultante.

Questão 3.1 — *Prova 2009-A — Q3-A*

Apresente a função de custo do algoritmo abaixo para determinar o número de vezes que `faz_algo` é executada:

```
VERIFICA_ALGO(int n) {
    if(n > 0) {
        FOR(x=0; x < n; x++) {
            FOR(y=0; y < x; y++) { faz_algo; }
        }
        VERIFICA_ALGO(n-1);
    }
}
```

Resolução Passo a Passo:

1. **Trabalho no Nível** n: Os laços aninhados executam $\sum_{x=0}^{n-1} x = \frac{n(n-1)}{2}$ vezes.
2. **Equação de Recorrência**: $T(n) = T(n-1) + \frac{n(n-1)}{2}$ com $T(0) = 0$.
3. **Expansão por Iteração**:

$$T(n) = \sum_{k=1}^{n} \frac{k(k-1)}{2} = \frac{1}{2}\left(\sum_{k=1}^{n} k^2 - \sum_{k=1}^{n} k\right)$$

4. **Substituição das Fórmulas de Somatório**:

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

$$T(n) = \frac{1}{2}\left(\frac{2n^3 + 3n^2 + n - 3n^2 - 3n}{6}\right) = \frac{n^3 - n}{6}$$

5. **Resultado**: Função de Custo Exata: $T(n) = \frac{n^3-n}{6}$, Classe: $\mathbf{\Theta(n^3)}$.

Questão 3.2 *p1-2024-2-es-segunda-chamada — Q1*

Calcule e apresente o número de vezes que a `operacao-unitaria` será executada no código abaixo em função de n:

```
int funcao(int n) {
    if(n > 1) {
        funcao(n/4);
        for(int x = 0; x < n; x++) { operacao-unitaria; }
        for(int x = 0; x < n; x++) { operacao-unitaria; }
        return 4*funcao(n/4);
    } else { operacao-unitaria; }
    return 24;
}
```

Resolução Passo a Passo:

1. **Contagem das Chamadas Recursivas e Laços**: O código faz 2 chamadas recursivas a `funcao(n/4)` e executa dois laços `for` que juntos realizam $n + n = 2n$ operações unitárias.
2. **Equação de Recorrência**: $T(n) = 2T(n/4) + 2n$ para $n > 1$, e $T(1) = 1$.
3. **Aplicação do Teorema Mestre**:
   - $a = 2, b = 4, f(n) = 2n$.
   - $n^{\log_b a} = n^{\log_4 2} = n^{0.5} = \sqrt{n}$.
   - Como $f(n) = 2n = \Omega(n^{0.5+0.5})$, aplica-se o **Caso 3**.
   - Teste de Regularidade: $af(n/b) = 2 \cdot (2(n/4)) = n \leq c(2n)$ para $c = 1/2 < 1$.
4. **Resultado**: Classe de Complexidade: $\mathbf{\Theta(n)}$.

Questão 3.3 *p1-ES-Substitutiva — Q2*

Calcule e apresente o número de vezes que a `operacao-unitaria` será executada pelo método da iteração:

```
int funcao(int n) {
    operacao-unitaria;
    if(n > 0) {
        return funcao(n-1) + funcao(n-1);
    }
    return 4;
}
```

Resolução Passo a Passo por Iteração:

1. **Recorrência**: $T(n) = 2T(n-1) + 1$ para $n > 0$, com $T(0) = 1$.
2. **Passo 1**: $T(n) = 2T(n-1) + 1$.

3. **Passo 2**: $T(n) = 2(2T(n-2)+1)+1 = 2^2T(n-2)+2+1$.
4. **Passo 3**: $T(n) = 2^2(2T(n-3)+1)+2+1 = 2^3T(n-3)+2^2+2+1$.
5. **Passo k**: $T(n) = 2^kT(n-k)+\sum_{i=0}^{k-1}2^i = 2^kT(n-k)+(2^k-1)$.
6. **Caso Base**: $n-k=0 \implies k=n$.

$$T(n) = 2^nT(0)+(2^n-1) = 2^n(1)+2^n-1 = 2\cdot 2^n-1 = 2^{n+1}-1$$

7. **Resultado**: Custo Exato: $T(n) = 2^{n+1}-1$, Classe: $\mathbf{\Theta(2^n)}$.

Questão 3.4 *p1-ES-Substitutiva — Q4*

Apresente a classe de complexidade utilizando o Teorema Mestre:

a) $T(n) = 2T(n/2)+2n$ b) $T(n) = 3T(n/2)+1$

Resolução Passo a Passo:

1. **Item a**: $a=2, b=2, f(n)=2n$. $n^{\log_2 2} = n^1$. Como $f(n) = \Theta(n^1)$, aplica-se o **Caso 2**.

$$T(n) = \mathbf{\Theta(n\log n)}$$

2. **Item b**: $a=3, b=2, f(n)=1$. $n^{\log_2 3} \approx n^{1.585}$. Como $f(n) = 1 = O(n^{\log_2 3-0.1})$, aplica-se o **Caso 1**.

$$T(n) = \mathbf{\Theta(n^{\log_2 3})}$$

4 Grupo 4: Programação Dinâmica

Guia Passo a Passo Geral — Programação Dinâmica

Para resolver problemas e construir algoritmos de Programação Dinâmica:

1. **Seguir os 4 Passos Clássicos do Paradigma**:
   (a) **Subestrutura Ótima**: Mostrar que a solução ótima do problema contém soluções ótimas de subproblemas.
   (b) **Relação de Recorrência**: Expressar o valor da solução em termos de subproblemas menores.
   (c) **Preenchimento Bottom-Up**: Construir a tabela interativamente.
   (d) **Reconstrução da Solução**: Rastrear a matriz para recuperar o resultado.
2. **Construção da Matriz DP (Tabela $(m+1) \times (n+1)$)**:
   - Preencher a linha 0 e a coluna 0 com os valores do caso base.
   - Preencher as células (i, j) linha por linha aplicanto a equação de recorrência.

Questão 4.1 *prova1-2024-ES — Q2*

Qual a sequência de passos a ser seguida no projeto de um algoritmo de programação dinâmica?

Resolução Passo a Passo:

1. **Caracterizar a Estrutura de uma Solução Ótima**: Verificar se o problema possui subestrutura ótima (solução global é composta por soluções ótimas locais) e subproblemas sobrepostos.
2. **Definir Recursivamente o Valor da Solução**: Formular a equação de recorrência para o custo do subproblema.
3. **Computar o Valor Bottom-Up**: Preencher uma tabela (vetor/matriz) resolvendo os menores subproblemas primeiro e armazenando os resultados.
4. **Construir a Solução Ótima**: Rastrear a tabela para recuperar a sequência de decisões que levou ao valor ótimo.

Questão 4.2 *prova1-2024-ES-Substitutiva — Q4*

Mostre a matriz de programação dinâmica para a Maior Subsequência Comum (LCS) entre **"BABA"** e **"ADABA"**.

Resolução Passo a Passo:

1. **Cadeias**: $X =$ "BABA" $(m = 4)$ e $Y =$ "ADABA" $(n = 5)$.
2. **Equação de Recorrência do LCS**:

$$L[i,j] = \begin{cases} 0 & \text{se } i = 0 \text{ ou } j = 0 \\ L[i-1, j-1] + 1 & \text{se } X[i] = Y[j] \\ \max(L[i-1, j], L[i, j-1]) & \text{se } X[i] \neq Y[j] \end{cases}$$

3. **Matriz DP Resultante**:

$L[i,j]$	$\emptyset$	**A**	**D**	**A**	**B**	**A**
$\emptyset$	0	0	0	0	0	0
B	0	0	0	0	1	1
A	0	1	1	1	1	2
B	0	1	1	1	2	2
A	0	1	1	2	2	**3**

4. **Resultado**: Comprimento da LCS = **3**. Uma subsequência resultante é **"ABA"**.

Questão 4.3 *p1-ES-Substitutiva — Q3*

Apresente uma FUNÇÃO em C utilizando programação dinâmica para implementar a solução recursiva da comparação de strings $D[X_m, Y_n]$.

Resolução Passo a Passo em Código C:

```c
int distancia_dp(char X[], char Y[], int m, int n) {
    int D[m+1][n+1];

    for(int i = 0; i <= m; i++) D[i][0] = i;
    for(int j = 0; j <= n; j++) D[0][j] = j;

    for(int i = 1; i <= m; i++) {
        for(int j = 1; j <= n; j++) {
            if(X[i-1] == Y[j-1]) {
                D[i][j] = D[i-1][j-1];
            } else {
                int min_val = D[i-1][j];
                if(D[i][j-1] < min_val) min_val = D[i][j-1];
                if(D[i-1][j-1] < min_val) min_val = D[i-1][j-1];
                D[i][j] = 1 + min_val;
            }
        }
    }
    return D[m][n];
}
```

5 Grupo 5: Teoria de Algoritmos, Estruturas de Dados e Estimativa Empírica

Guia Passo a Passo Geral — Extrapolação Empírica e Teoria

Para resolver questões teóricas e estimativas de tempo de execução:

1. **Para Estimativas de Tempo Empírico**:
   (a) Identificar a classe de complexidade $f(n)$ dada na questão (ex: $f(n) = \log_2 n$ ou $f(n) = n$).
   (b) Relacionar o tempo de execução $T(n)$ com a ordem de crescimento: $T(n) = c \cdot f(n)$.
   (c) Calcular a razão proporcional entre os dois tamanhos de entrada n_1 e n_2:

$$T(n_2) = T(n_1) \cdot \frac{f(n_2)}{f(n_1)}$$

2. **Para Questões Teóricas de Divisão e Conquista**:
   - Explicar os três passos: Dividir, Conquistar e Combinar.
   - Lembrar que o Teorema Mestre não se aplica a todas as recorrências (requer subproblemas de tamanhos iguais n/b).

Questão 5.1 *p1-ES-Substitutiva — Q1*

Uma árvore vermelho e preto com 2^{20} chaves teve tempo médio de 1 ns para buscas. Apresente uma estimativa do tempo médio esperado para 2^{60} chaves.

Resolução Passo a Passo:

1. **Complexidade da Busca**: Em uma Árvore Vermelho e Preto, o tempo de busca é proporcional à altura da árvore, ou seja, $T(N) = c \cdot \log_2 N$.
2. **Cálculo da Constante c para $N_1 = 2^{20}$**:

$$T(2^{20}) = c \cdot \log_2(2^{20}) = c \cdot 20 = 1 \text{ ns} \implies c = \frac{1}{20} \text{ ns}$$

3. **Cálculo do Tempo Esperado para $N_2 = 2^{60}$**:

$$T(2^{60}) = c \cdot \log_2(2^{60}) = \frac{1}{20} \cdot 60 = 3 \text{ ns}$$

4. **Resultado**: Tempo estimado: **3 nanosegundos**.

Questão 5.2 *prova1-2026-Férias — Q5*

Um algoritmo de custo $\Theta(\log n)$ executou $n = 10.000$ em 1 ms. Qual o tempo esperado para $n = 100.000.000$?

Resolução Passo a Passo:

1. **Relação Funcional**: $T(n) = c \cdot \log_{10} n$.

2. **Entrada 1** ($n_1 = 10^4$): $T(10^4) = c \cdot \log_{10}(10^4) = 4c = 1$ ms $\implies c = 0.25$ ms.

3. **Entrada 2** ($n_2 = 10^8$): $T(10^8) = c \cdot \log_{10}(10^8) = 8c = 8 \cdot 0.25$ ms $= 2$ ms.

4. **Resultado**: Tempo estimado: **2 milissegundos**.

Questão 5.3 ***Prova 2009-A — Q2***

Compare QuickSort e HeapSort em termos de tempo de execução (pior e médio caso) e diga quando o HeapSort pode ser melhor.

Resolução Passo a Passo:

1. **Complexidade de Tempo**:
   - **QuickSort**: Caso Médio $\Theta(n \log n)$, Pior Caso $\Theta(n^2)$ (quando o pivô escolhido é o mínimo/máximo).
   - **HeapSort**: Caso Médio $\Theta(n \log n)$, Pior Caso $\Theta(n \log n)$ garantido.
2. **Quando o HeapSort é uma Escolha Melhor**:
   - Em sistemas de tempo real ou aplicações críticas onde o pior caso quadratico $O(n^2)$ não pode ser tolerado sob nenhuma hipótese.
   - Em ambientes com restrição severa de memória, pois o HeapSort é estritamente $O(1)$ in-place, enquanto o QuickSort requer pilha de recursão $O(\log n)$.